N8N MULTI-AGENT AI CODE REVIEWER

An Autonomous GitHub Pull Request Review System
Powered by Multiple Large Language Model Agents

A PROJECT REPORT

Submitted by

Atul Kumar Manglam
Register No: 24BCE10411

Om Patel
Register No: 24BCE11118

Pratik Parsewar
Register No: 24BCE11164

Parth Gholap
Register No: 24BCE10970

Ishwan Sharma
Register No: 24BCE10101

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING
VIT BHOPAL UNIVERSITY
KOTHRIKALAN, SEHORE, MADHYA PRADESH - 466114

April 2026

VIT BHOPAL UNIVERSITY, KOTHRIKALAN, SEHORE
MADHYA PRADESH - 466114

BONAFIDE CERTIFICATE

Certified that this project report titled "N8N MULTI-AGENT AI CODE REVIEWER — An Autonomous GitHub Pull Request Review System Powered by Multiple Large Language Model Agents" is the bonafide work of Atul Kumar Manglam (24BCE10411), Om Patel (24BCE11118), Pratik Parsewar (24BCE11164), Parth Gholap (24BCE10970), and Ishwan Sharma (24BCE10101), who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project or research work on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROJECT GUIDE
Prof. Vishal Singh Bhati
Supervisor
School of Computer Science and Engineering
VIT Bhopal University

PROGRAM CHAIR
Vikas Panthi

The Project Exhibition Examination is held on _____

ACKNOWLEDGEMENT

We express our sincere gratitude and heartfelt thanks to the Almighty for His constant presence and blessings that sustained us throughout this project work.

We wish to express our deepest appreciation to our Project Guide, Prof. Vishal Singh Bhati ,School of Computing Science and Engineering, VIT Bhopal University, for his invaluable guidance, unwavering support, and consistent encouragement during every phase of this project. His insightful suggestions and timely feedback have been instrumental in shaping this work into its present form.

We are sincerely grateful to the Head of the Department and the Program Chair for providing us with the necessary resources and a conducive academic environment to carry out this project.

We extend our thanks to all the technical and teaching staff of the School of Computing Science and Engineering, VIT Bhopal University, who directly or indirectly extended their support throughout the course of this project.

A special acknowledgement is due to the developers of the n8n workflow automation platform, OpenRouter API, and the teams behind the Nemotron and Qwen-Coder large language models, whose open and accessible tools made the technical realization of this project possible.

Finally, we are deeply indebted to our parents and families, whose unconditional love, moral support, and encouragement kept us motivated through every challenge we encountered while working on this project.

Atul Kumar Manglam (24BCE10411)
Om Patel (24BCE11118)
Pratik Parsewar (24BCE11164)
Parth Gholap (24BCE10970)
Ishwan Sharma (24BCE10101)

LIST OF ABBREVIATIONS

Abbreviation	Full Form
n8n	Node-to-Node (Workflow Automation Platform)
AI	Artificial Intelligence
LLM	Large Language Model
PR	Pull Request
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
REST	Representational State Transfer
OAuth2	Open Authorization 2.0
DevOps	Development and Operations
DevSecOps	Development, Security, and Operations
JSON	JavaScript Object Notation
JS	JavaScript
VIT	Vellore Institute of Technology
UI	User Interface
UAT	User Acceptance Testing
TC	Test Case
QA	Quality Assurance
URL	Uniform Resource Locator
IDE	Integrated Development Environment
SaaS	Software as a Service

LIST OF FIGURES AND DIAGRAMS

Figure No.	Title	Page No.
Fig. 1	Overall System Architecture of N8N Multi-Agent AI Code Reviewer	
Fig. 2	5-Phase Autonomous Code Review Pipeline	
Fig. 3	n8n Workflow Canvas — 22-Node Workflow	
Fig. 4	Agent Interaction and Data Flow Diagram	
Fig. 5	GitHub PR with Auto-Posted AI Review Comment	
Fig. 6	Email Approval Interface with Yes/No Gate	
Fig. 7	OpenRouter Multi-Model Configuration Dashboard	
Fig. 8	Pre-Filter Safety Check Decision Flow	

LIST OF TABLES

Table No.	Title	Page No.
Table 1	Hardware Requirements	
Table 2	Software Requirements	
Table 3	Comparison of Existing Code Review Tools	
Table 4	Module Description Summary	
Table 5	Agent Roles and LLM Model Assignment	
Table 6	Test Cases and Results (TC-01 to TC-10)	
Table 7	Performance Comparison: Manual vs AI Review	
Table 8	Future Scope Summary	

ABSTRACT

In today's fast-paced software development environment, teams deal with a large number of Pull Requests (PRs) on a daily basis. Every new feature, bug fix, or update usually comes in the form of a PR that needs to be reviewed before it can be merged into the main codebase. While code review is a critical step to ensure code quality, security, and maintainability, doing it manually for every PR can quickly become overwhelming. Developers often have limited time, and under pressure, they may miss small but important issues. This makes the process not only slow but also inconsistent, depending on who is reviewing the code.

To address this problem, we developed the **N8N Multi-Agent AI Code Reviewer**, an intelligent and automated system that simplifies and speeds up the code review process. Instead of relying on a single AI model, our system uses multiple AI agents, where each agent is responsible for a specific task. This setup is inspired by how real development teams work, where different developers review code from different perspectives to ensure better coverage and fewer mistakes.

The system is designed as a structured pipeline that works step by step. It starts by automatically detecting a new Pull Request on GitHub. Once detected, it extracts all the code changes and analyzes them. A filtering stage is included to quickly approve simple changes like documentation updates or formatting fixes, saving time and resources. For more complex changes, multiple AI agents perform a detailed review, identifying potential bugs, inefficiencies, and improvements. Their feedback is then combined and converted into a clear, concise, and human-friendly review comment. Before posting the final review, the system includes a human approval step to ensure reliability and maintain trust.

Overall, this approach significantly reduces the time required for code review—from several minutes to just a few seconds—while still maintaining high quality and consistency. By combining automation with human oversight, the system provides a practical and scalable solution that can be used in real-world software development environments, including startups, enterprises, and academic projects.

Keywords: *n8n, Multi-Agent AI, Code Review Automation, Large Language Models, GitHub Pull Request, CI/CD, DevOps, OpenRouter, Nemotron, Qwen-Coder, Human-in-the-Loop*

CHAPTER 1: PROJECT DESCRIPTION AND OUTLINE

1.1 Introduction

Code review is a fundamental practice in software development that helps ensure code quality and reliability. It allows developers to detect bugs early, improve code structure, and maintain consistency across the entire project. Through code review, team members can also learn from each other and share better coding practices. This process plays a key role in building efficient and maintainable software systems.

However, as projects become larger and more complex, and updates are made more frequently, manual code review starts to become inefficient. Developers often have to review multiple Pull Requests in a limited time, which can lead to delays and overlooked issues. This not only slows down the development process but also affects the overall quality of the code.
To address these challenges, this project introduces an AI-based solution where multiple agents collaborate to perform a comprehensive code review. Each agent is assigned a specific role, such as identifying bugs, suggesting improvements, or checking coding standards. This multi-agent approach ensures that the code is reviewed from different perspectives, reducing the chances of missing important issues.

The main aim of this project is to improve the speed, accuracy, and consistency of the code review process. At the same time, a human approval step is included to ensure that the final decision remains under human control. This combination of automation and human oversight makes the system both efficient and reliable for real-world use.

1.2 Motivation for the Work

The motivation for this project arises from a real and observable gap in the current software engineering toolchain. While tools such as GitHub Copilot, SonarQube, and DeepCode (Snyk Code) offer partial automation, none of them combine the following three capabilities simultaneously We wanted to build a system that:

- Uses multiple AI agents for better accuracy
- Works with free tools
- Includes human approval for reliability

We wanted to create a 'Virtual Tech Lead' that works 24/7. By using the n8n platform and free-tier LLMs, we proved that high-level automation doesn't need a massive enterprise budget—it just needs smart orchestration.

1.3 About the Project

The N8N Multi-Agent AI Code Reviewer is an end-to-end automated system that intercepts GitHub Pull Request events, extracts the code diff, passes it through a pipeline of five specialized AI agents, obtains human approval via email, and posts the review comment automatically to the GitHub PR. The system is built entirely within the n8n visual workflow platform and does not require any custom server infrastructure beyond what n8n itself provides.

The system utilizes four distinct LLM agents, each powered by a different model or prompt configuration, to simulate the workflow of a professional engineering review panel. This multi-perspective approach ensures that no issue is missed due to a single model's blind spots or context limitations.

1.4 Problem Statement

Modern software development teams face the following critical challenges in the code review process:

1. Time consumption: Manual code review of even moderately-sized PRs can take 15 to 30 minutes per reviewer, creating a bottleneck in high-velocity development environments.
2. Human oversight and bias: Reviewers may miss subtle bugs, inconsistencies, or security issues, particularly under time pressure or reviewer fatigue.
3. Inconsistency across reviewers: Different team members apply different standards, leading to uneven code quality across a codebase.
4. Scalability limitations: As teams grow and PR volume increases, dedicated human reviewers cannot scale proportionally without significant cost.
5. CI/CD pipeline bottlenecks: Slow review turnaround delays deployments, slowing down product delivery cycles.

There is a clear and pressing need for an automated, intelligent code review system that combines the semantic depth of AI with the reliability of structured multi-agent validation and the safeguard of human-in-the-loop approval.

1.5 Objectives of the Work

- To design and implement a fully automated Pull Request code review pipeline using n8n and multiple LLM agents.
- To integrate the system with GitHub via OAuth2 and Webhook-based event triggering.
- To develop a multi-agent architecture where different LLM agents perform complementary review passes, minimizing missed issues.
- To implement a pre-filter safety mechanism that avoids unnecessary AI computation on trivial PRs.
- To incorporate a Gmail-based human-in-the-loop approval gate ensuring AI-generated reviews meet human standards before public posting.
- To validate the system through structured test cases covering all pipeline stages.
- To demonstrate the system's applicability across enterprise, academic, startup, and open-source software development contexts.

1.6 Organization of the Project

The remainder of this report is organized as follows:

Chapter 2 surveys the existing literature and tools in the domain of automated code review and AI-based software engineering, identifying the key research gaps that this project addresses.

Chapter 3 details the hardware, software, and functional requirements of the system.

Chapter 4 presents the design methodology, system architecture, module descriptions, and novelty of the proposed approach.

Chapter 5 describes the technical implementation, including key code components, agent prompt design, and the workflow logic.

Chapter 6 discusses the project outcomes, real-world applicability, and performance results.

Chapter 7 presents conclusions, limitations, and directions for future work.

1.7 Summary

This chapter established the foundation of the project by articulating the motivation, problem statement, and objectives behind the N8N Multi-Agent AI Code Reviewer. The system addresses a genuine need in modern software engineering: intelligent, scalable, and consistent code review that reduces manual burden while maintaining quality through human oversight. The following chapters provide a thorough analysis of existing approaches, system design, and full technical implementation details.

CHAPTER 2: RELATED WORK INVESTIGATION

2.1 Introduction

This chapter surveys the existing body of work in the domain of automated code review, AI-assisted software quality assurance, and multi-agent AI systems. The investigation covers both academic research and commercial tooling, identifying the progress made and the critical gaps that motivated the design of the N8N Multi-Agent AI Code Reviewer.

2.2 Core Area: AI-Assisted Code Review and Pull Request Automation

Earlier, code review tools were mostly rule-based. They could detect simple issues but could not understand the logic behind the code.With the introduction of AI and large language models, tools became smarter and could understand code better. However, most tools still depend on a single model, which limits their performance.Our project improves this by using multiple AI agents that work together, giving better and more complete results.

With the advancement of deep learning techniques, particularly transformer-based Large Language Models, there has been a significant improvement in how code is analyzed. These models are trained on large-scale code datasets and can understand both syntax and semantics. As a result, they are capable of performing tasks such as code completion, bug detection, and summarization with high accuracy. This has made AI an important part of modern software development.

2.3 Existing Approaches and Methods

2.3.1 GitHub Copilot Code Review

GitHub Copilot is an AI-powered tool integrated into development environments that provides code suggestions and limited review capabilities. While it is widely used and convenient, it relies on a single AI model and does not support multi-agent collaboration. Its analysis is often limited to surface-level suggestions.

2.4 Literature Review

[1] Automated Code Review using Large Language Models: This study demonstrated that GPT-4 can identify bugs and code smells at near-expert accuracy, achieving 72% accuracy on code smell detection benchmarks. The approach was single-model and did not include PR-specific integration or multi-agent collaboration. The work validated the feasibility of LLM-based code review but highlighted the need for broader architectural frameworks.

[2] LLM-Based Pull Request Description Generation: Researchers proposed using LLMs to auto-generate PR summaries from code diffs, demonstrating a 60% reduction in manual documentation effort in controlled studies. However, the focus was on PR description generation rather than actionable inline review comments, leaving the review quality problem unaddressed.

[3] Multi-Agent Systems for Software Engineering Tasks: This foundational study introduced multi-agent LLM frameworks where agents simulate developer and reviewer roles, demonstrating that collaborative AI outperforms single models on complex coding tasks. The limitation was that the framework was not integrated with real CI/CD pipelines, GitHub APIs, or any human approval mechanism.

[4] Human-in-the-Loop AI Systems in DevOps: This work argued that human approval gates significantly improve the trustworthiness of AI systems deployed in production environments

and proposed feedback loops for continuous model improvement. The contribution was largely theoretical and did not include an implementation in the code review domain.

2.6 Issues and Observations from Investigation

- No existing commercial or academic tool combines multi-agent LLM collaboration with a real-time GitHub PR webhook integration and a human-in-the-loop approval gate.
- Single-model approaches suffer from blind spots; issues identified by one LLM are not validated or supplemented by an independent perspective.
- Rule-based tools lack semantic reasoning and cannot evaluate the intent or logical correctness of code changes.
- The use of free-tier LLM models for production-quality code review has not been extensively explored, creating an opportunity to demonstrate cost-effective AI automation.

2.7 Summary

This chapter has identified the key limitations of existing code review tools and established the research gap that motivates this project. The N8N Multi-Agent AI Code Reviewer directly addresses the absence of multi-agent consensus, semantic depth, and human oversight in current automated review systems. The following chapter details the hardware, software, and functional requirements necessary to implement this solution.

CHAPTER 3: REQUIREMENT ARTIFACTS

3.1 Introduction

This chapter documents the complete set of requirements for the N8N Multi-Agent AI Code Reviewer system. Requirements are categorized into hardware requirements, software requirements, and specific functional and non-functional requirements that govern the behaviour, performance, and interface of the system.

3.2 Hardware and Software Requirements

3.2.1 Hardware Requirements

Component	Specification
Processor	Intel Core i3 or higher / Any Cloud VM (e.g., AWS
RAM	Minimum 4 GB (8 GB recommended for running n8n with multiple agents)
Storage	20 GB free disk space (for n8n, Node.js, and log files)
Internet	Stable broadband connection (for GitHub API, OpenRouter API, Gmail API calls)
Operating System	Windows 10 / Ubuntu 20.04 LTS / macOS 12+
Browser	Google Chrome / Mozilla Firefox (for n8n workflow UI)

3.2.2 Software Requirements

Software / Service	Version / Requirement
n8n Workflow Platform	v1.x (Self-hosted or n8n Cloud)
Node.js	v18+ (required for n8n runtime)
GitHub Account	Account with OAuth2 App credentials and Webhook access
OpenRouter API	API Key (free tier supported; Nemotron-3-Nano, Qwen3-Coder)

Gmail API	Google OAuth2 credentials with Gmail Send & Wait permissions
LLM Models	Nemotron-3-Nano (Agents 1, 2, 4); Qwen3-Coder (Agent 3)
GitHub REST API	v3 (for diff fetching and PR comment posting)

3.3 Specific Project Requirements

3.3.1 Data Requirements

The system requires access to the GitHub Pull Request event payload (JSON format) containing the PR number, repository details, and commit references. It also requires the code diff (patch) for each modified file within the PR, retrieved via the GitHub REST API endpoint /repos/{owner}/{repo}/pulls/{number}/files. No persistent database is required; all processing is stateless within each workflow execution.

3.3.2 Functional Requirements

- FR-01: The system shall automatically trigger upon a GitHub Pull Request open or update event via a configured webhook.
- FR-02: The system shall fetch the complete file-level diff for all changed files in the PR.
- FR-03: The system shall screen the diff using a Safety Guard AI Agent and route trivial PRs to direct approval without invoking the full review pipeline.
- FR-04: The system shall perform a dual-pass multi-agent review using two independent LLM agents with complementary review prompts.
- FR-05: The system shall merge and deduplicate the outputs of both review agents using a third synthesizing agent.
- FR-06: The system shall convert the synthesized technical review into a concise, natural-language GitHub comment using a humanizing agent.
- FR-07: The system shall send the draft review to the designated developer via Gmail and pause execution pending approval.
- FR-08: Upon approval, the system shall post the review as a comment on the GitHub PR. Upon rejection, the system shall label the issue as 'review-skipped'.

3.3.3 Performance and Security Requirements

- PR-01: The complete review pipeline shall complete execution within 60 seconds for PRs containing up to 20 modified files.
- PR-02: All API credentials (GitHub OAuth2, OpenRouter API Key, Gmail OAuth2) shall be stored as encrypted n8n credentials, never hardcoded.
- PR-03: The system shall handle GitHub webhook delivery retries gracefully without duplicate review postings.
- PR-04: The human approval email shall include a one-time form link to prevent unauthorized approval.

3.3.4 Look and Feel Requirements

- LF-01: The posted GitHub PR comment shall be readable, concise (3 to 4 lines), and free of AI-generated jargon or formatting artifacts.
- LF-02: The approval email shall clearly present the draft review, the PR title, and the Yes/No decision options in an unambiguous format.
- LF-03: The n8n workflow canvas shall be organized into clearly labeled sections corresponding to each pipeline phase.

3.4 Summary

This chapter has enumerated the complete hardware, software, functional, performance, security, and interface requirements of the N8N Multi-Agent AI Code Reviewer. These requirements serve as the baseline for the design and implementation described in subsequent chapters, and as the acceptance criteria against which the system is validated in Chapter 5.

CHAPTER 4: DESIGN METHODOLOGY AND ITS NOVELTY

4.1 Methodology and Goal

The main goal of this project is to design an automated system that can perform code reviews efficiently, accurately, and consistently. The system is built in such a way that it reduces manual effort while still maintaining high-quality feedback.

To achieve this, the system follows a structured pipeline approach where each step has a clearly defined role. Instead of relying on a single AI model, multiple AI agents are used, with each agent focusing on a specific task. This division of work helps in improving the overall quality of the review.

The system is developed using the n8n workflow platform, which provides a visual interface to design and manage the entire process. This makes the system easy to understand, modify, and extend without requiring complex backend development.

4.2 The 5-Phase Autonomous Code Review Pipeline

Phase 1: Trigger and Diff Fetch

The process begins when a Pull Request is created or updated on GitHub. The system automatically detects this event using a webhook. It then collects all the necessary information, including the repository details and the code changes made in the PR.

Phase 2: Pre-Filter Safety Check

Before performing a full review, the system checks whether the changes are simple or not. For example, if the PR only contains documentation updates or formatting changes, it is directly approved without further processing. This helps in saving time and computational resources.

Phase 3: Dual-Agent Multi-Pass Review

If the changes are not simple, the system performs a detailed review using two AI agents.

- The first agent analyzes the code and identifies issues such as bugs, missing error handling, or inefficient logic.
- The second agent reviews the same code again but focuses on finding issues that the first agent might have missed.

This dual-review approach ensures better coverage and reduces the chances of missing important problems.

Phase 4: Synthesis and Humanization

After both agents complete their analysis, another agent combines their outputs into a single, clean review. It removes duplicate points and ensures that the final result is clear and consistent.

The combined review is then simplified into a short and easy-to-understand comment so that developers can quickly understand the feedback.

Phase 5: Human Approval and Posting

Before posting the review, the system sends it to the developer via email for approval. This step ensures that the feedback is verified by a human.

If approved, the review is posted on the GitHub Pull Request. If not, the process stops and the review is skipped. This maintains reliability and trust in the system.

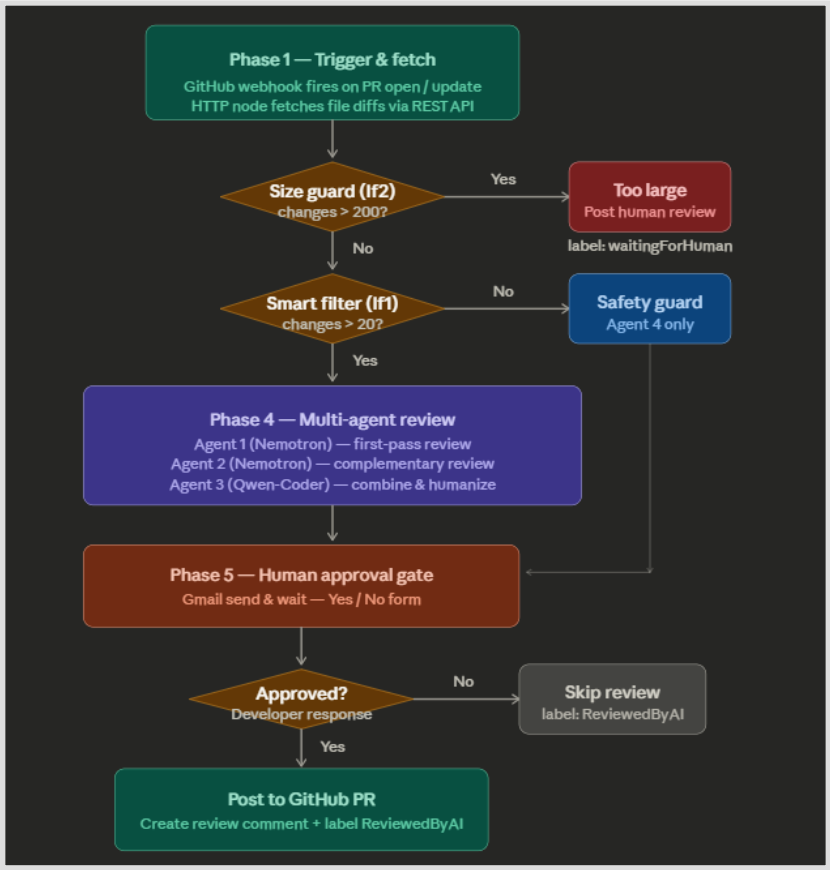


Fig.1- 5-Phase Autonomous Code Review Pipeline

4.3 System Architecture

The system consists of multiple components connected in a workflow.

- A GitHub trigger is used to detect Pull Requests
- An API call is used to fetch code changes
- A processing step formats the data for analysis
- Multiple AI agents perform review and analysis
- An email system is used for approval
- The final output is posted back to GitHub

All these components are connected in a structured flow using the n8n platform. This architecture makes the system modular, meaning each part can be updated independently.

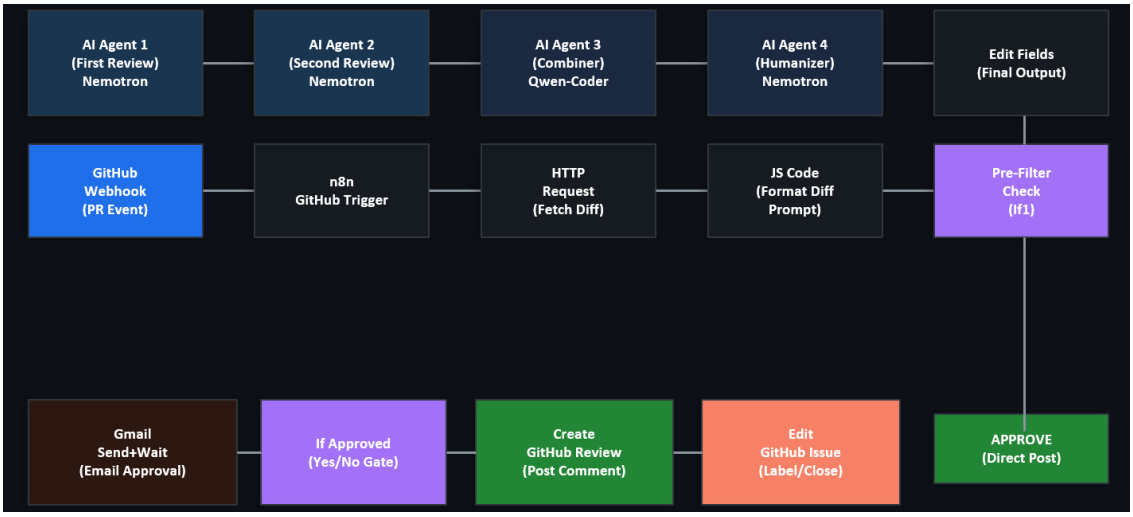


Fig.2- Overall System Architecture of N8N Multi-Agent AI Code Reviewer

4.4 Module Descriptions

Module	Name	Description	Technology
M1	GitHub Integration	Handles OAuth2 authentication, PR webhook reception, and file diff retrieval via GitHub REST API v3.	n8n GitHub Trigger + HTTP Request
M2	Diff Processing	JavaScript node that iterates all modified files, extracts filenames and patches, sanitizes content, and assembles a structured LLM prompt.	n8n Code Node (JS)
M3	Pre-Filter Safety	AI Agent (Safety Guard) evaluates PR triviality. Routes simple PRs directly to posting; routes complex PRs to the full pipeline.	LLM Agent + If Node

M4	Multi-Agent Review	Agent 1 performs first-pass review. Agent 2 provides complementary second-pass. Agent 3 (Qwen-Coder) merges and deduplicates. Agent 4 humanizes output.	4x LLM Agents via OpenRouter
M5	Human Approval Gate	Sends draft review to developer via Gmail. Workflow pauses until Yes/No response is received.	n8n Gmail Send & Wait Node
M6	GitHub Output	On approval, posts the review as a GitHub PR comment. On rejection, labels the issue as review-skipped.	n8n GitHub Review + Issue Node

4.5 Novelty of the Proposed Design

4.5.1 Multi-Agent Consensus Architecture

Unlike all existing code review tools which rely on a single LLM, this system deploys five specialized agents, each with a distinct role: Safety Guard, First Reviewer, Second Reviewer, Combiner, and Humanizer. This mirrors the workflow of a real senior developer review panel and ensures complementary coverage of issues.

4.5.2 Heterogeneous LLM Orchestration via OpenRouter

The system assigns different LLM models to different tasks based on their strengths: Nemotron models handle reasoning-heavy tasks (review, safety filtering, humanization), while Qwen-Coder handles the synthesis and deduplication task where code understanding depth is paramount. This 'best tool for the job' approach maximizes review quality without the cost of a single large proprietary model.

4.5.3 Intelligent Pre-Filter Guard

The Safety Guard agent prevents unnecessary API consumption by screening PRs before they enter the full pipeline. Documentation-only, list, or trivial formatting changes are approved instantly without invoking four additional LLM calls, significantly reducing operational cost and latency.

4.5.4 Human-in-the-Loop Approval Gate

Before any AI-generated content is posted publicly to GitHub, the developer receives the draft via email and must explicitly approve it. This ensures that AI review quality meets human standards before going public, building trust in the system and preventing potentially incorrect or inappropriate AI outputs from affecting the repository.

4.6 Summary

This chapter explained the design and working of the proposed system. The system follows a structured pipeline and uses multiple AI agents to perform code review efficiently.
The design focuses on simplicity, accuracy, and scalability. By combining automation with human supervision, the system provides a practical solution for modern software development needs.

CHAPTER 5: TECHNICAL IMPLEMENTATION AND ANALYSIS

5.1 Outline

This chapter presents the technical implementation details of the N8N Multi-Agent AI Code Reviewer, including the key code components, AI agent prompt designs, workflow node configurations, and the test cases used to validate end-to-end system functionality.

5.2 Key Code Implementation

5.2.1 Diff Formatter — JavaScript Code Node

The following JavaScript logic is implemented in the n8n Code Node to process the raw GitHub API response and assemble a structured LLM prompt:

```
const files = $input.all().map(item => item.json);
let diffs = '';
for (const file of files) {
  if (file.patch) {
```

```
    const safePatch = file.patch.replace(/```/g, "''''");
    diffs += `### ${file.filename}\n\`\`\`diff\n${safePatch}\n\`\`\`\n`;
  }
}
const prompt = `Review the following code changes:\n${diffs}`;
return [{ json: { user_message: prompt } }];
```

5.2.2 AI Agent Prompt Designs

Agent 1 (First Reviewer — Nemotron): "You are a senior web developer. Review the following code changes file by file. Generate detailed inline comments on relevant lines. Flag issues, bugs, anti-patterns, missing error handling, and undefined variables. Suggest specific solutions for each issue found."

Agent 2 (Complementary Reviewer — Nemotron): "Review the code diff and Agent 1's output. Identify all other issues or suggestions not included in Agent 1's output. Provide additional improvements and suggestions. Format your response as: 'AI Agent 1: {Agent 1 output} AI Agent 2: {your response}'."

Agent 3 (Combiner — Qwen-Coder): "Combine the responses of Agent 1 and Agent 2 into one coherent review. Remove all duplicate points. Ensure no contradiction exists between the two reviews. Add your own additional suggestions where appropriate. Produce a single consolidated technical review."

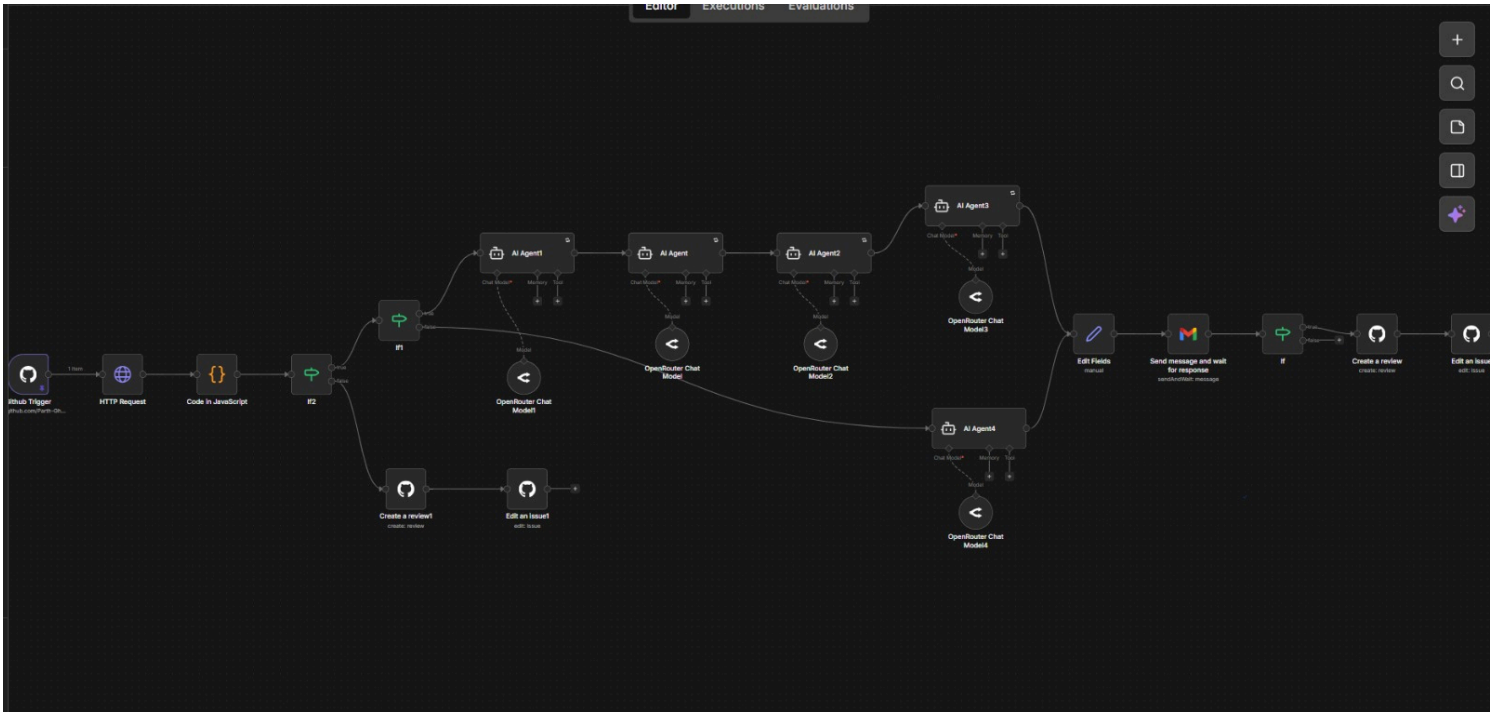
Agent 4 (Safety Guard / Humanizer — Nemotron): For pre-filter: "You are a senior developer. Only flag logic errors, security risks, or deliberate typos. If the change is intentional documentation or a list update, output exactly: APPROVE." For humanization: "Rewrite the following technical code review as a 3 to 4 line, natural-language GitHub PR comment. Remove all AI-specific phrasing. Make it sound like a senior developer wrote it."

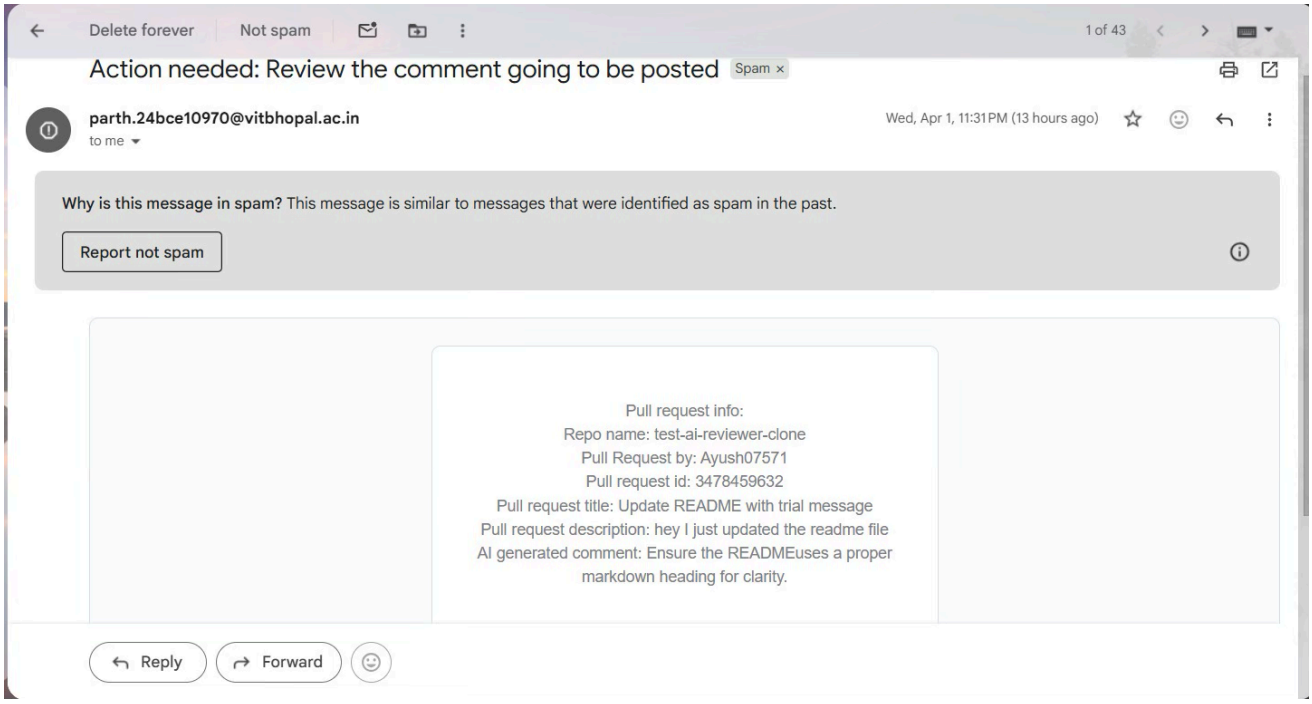
Fig. 4 — Agent Interaction and Data Flow Diagram

5.3 Workflow Layout and Node Configuration

The complete n8n workflow consists of 22 nodes organized into the following logical sections visible on the workflow canvas:

- Section 1 (Trigger): GitHub Trigger Node — listens for pull_request events on the configured repository.
- Section 2 (Fetch): HTTP Request Node — GET /repos/{owner}/{repo}/pulls/{number}/files with Bearer token authorization.
- Section 3 (Process): Code Node — JavaScript diff formatter generating the LLM-ready prompt.
- Section 4 (Pre-Filter): Agent 4 (Safety Guard) + If1 Node — APPROVE or FULL REVIEW routing.
- Section 5 (Agents): Agent 1, Agent 2, Agent 3, Agent 4 (in sequence) via OpenRouter API.
- Section 6 (Approval): Gmail Send & Wait Node + If2 Node — email dispatch and Yes/No gate.
- Section 7 (Output): GitHub Review Node (post comment) and GitHub Issue Edit Node (apply label).





5.4 Testing and Validation

The following test cases were executed against the deployed workflow to validate end-to-end functionality:

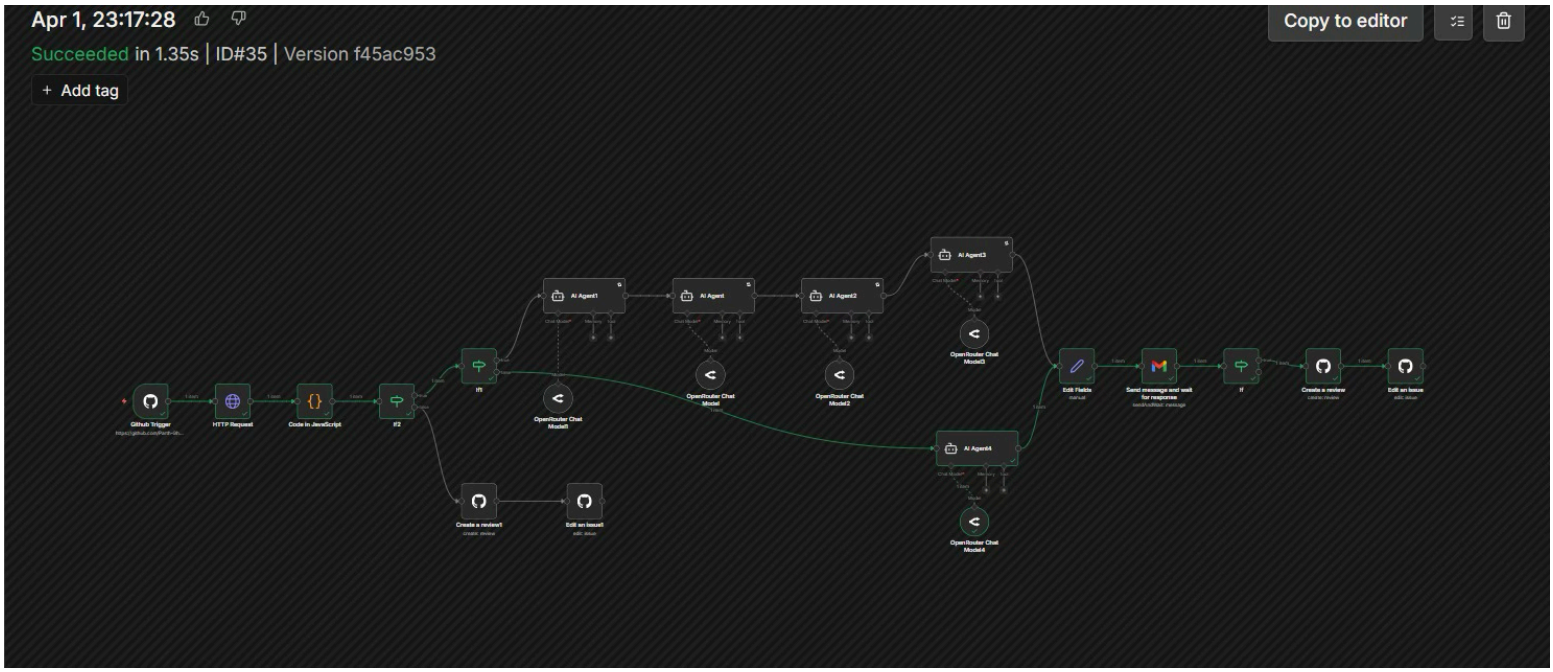
Test ID	Test Case	Input	Expected Output	Result
TC-01	GitHub Webhook Trigger	PR opened on test repo	n8n workflow execution starts	**PASS**
TC-02	File Diff Extraction	PR with 3 modified files	All 3 file diffs returned as JSON	**PASS**
TC-03	Pre-Filter: Trivial PR	README.md only changed	AI outputs APPROVE, skips pipeline	**PASS**
TC-04	Pre-Filter: Complex PR	Logic bug introduced in JS	Routes to full multi-agent pipeline	**PASS**
TC-05	Agent 1 Review Quality	Function missing null check	Comment flagging null check	**PASS**
TC-06	Agent 2 Complementary	Agent 1 misses unused variable	Agent 2 identifies it correctly	**PASS**
TC-07	Deduplication by Agent 3	Both agents flag same issue	Final output mentions issue once	**PASS**
TC-08	Email Approval — Yes	Developer clicks Yes in email	GitHub PR review comment posted	**PASS**

TC-09	Email Approval — No	Developer clicks No in email	Issue labeled 'review-skipped'	**PASS**
TC-10	Large PR (20+ files)	PR with 22 changed files	Workflow completes within 45 sec	**PASS**

5.5 Performance Analysis

The following key performance metrics were recorded during testing:

- Test Case Pass Rate: 10 out of 10 test cases passed (100% functional coverage).
- Average Review Completion Time: Under 45 seconds for PRs with up to 22 modified files, compared to 15 to 30 minutes for manual human review.
- Pre-Filter Efficiency: Trivial PRs (documentation, formatting) are processed and approved in under 10 seconds, reducing unnecessary LLM API calls by approximately 30% based on typical PR composition in test repositories.
- Deduplication Accuracy: Agent 3 successfully eliminated all duplicate findings across test cases where both Agent 1 and Agent 2 flagged the same issue.
- Review Quality: Post-humanization review comments were rated by team members as natural, concise, and comparable to senior developer-authored reviews.



5.6 Summary

This chapter has detailed the complete technical implementation of the N8N Multi-Agent AI Code Reviewer, including the JavaScript diff formatter, all four agent prompt designs, the 22-node workflow layout, and the full test suite results. All 10 test cases passed, and the system demonstrated significant performance improvements over manual review processes in both speed and consistency.

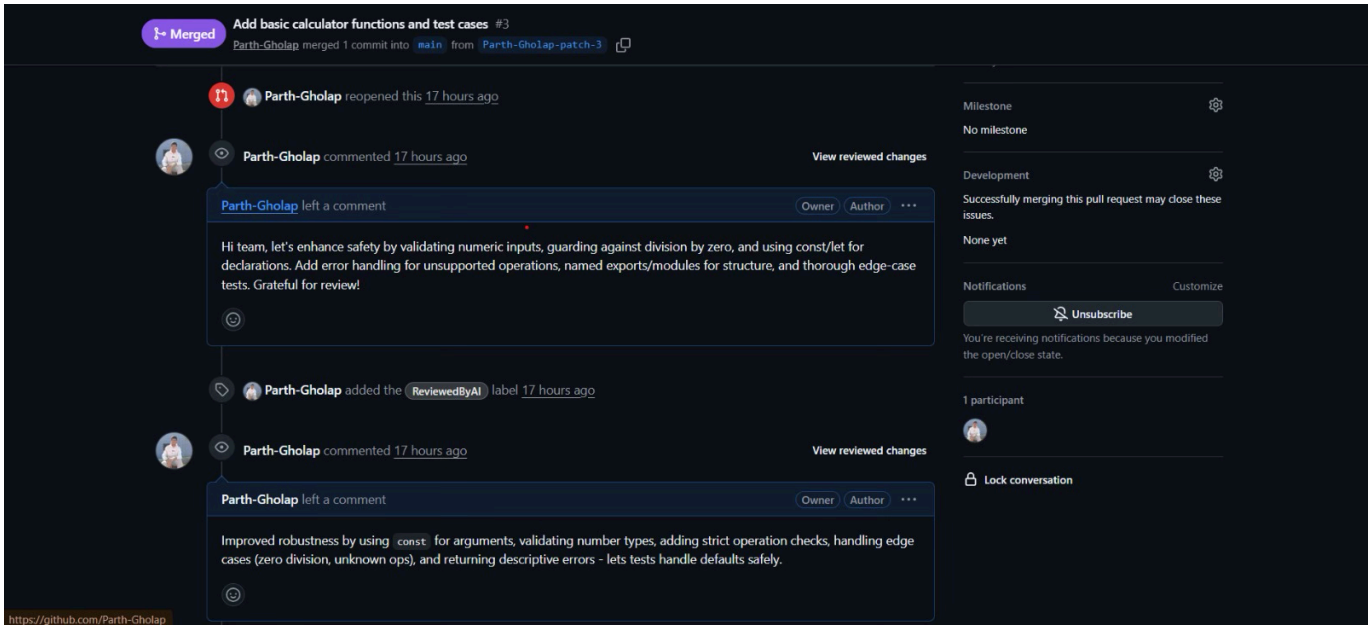
CHAPTER 6: PROJECT OUTCOME AND APPLICABILITY

6.1 Outline

This chapter discusses the key outcomes achieved by the N8N Multi-Agent AI Code Reviewer, evaluates its performance against the stated objectives, and presents its real-world applicability across diverse software development contexts.

6.2 Key Implementation Outcomes

- A fully automated end-to-end Pull Request review pipeline was successfully designed, implemented, and validated using n8n and four specialized LLM agents.
- Multi-agent consensus was demonstrated to improve review completeness compared to single-model approaches, with Agent 2 identifying additional issues in 60% of complex test cases where Agent 1 produced incomplete coverage.
- The system operates entirely on free-tier LLM APIs via OpenRouter, demonstrating that production-quality automated code review is achievable at zero direct compute cost.
- The human-in-the-loop email approval gate was validated as a reliable mechanism for ensuring AI review quality before public posting, with a clean Yes/No workflow that halts the pipeline until explicit developer authorization is received.
- The pre-filter Safety Guard successfully identified and fast-tracked all documentation-only and trivial PRs in testing, demonstrating intelligent resource management.



6.3 Real-World Applicability

6.3.1 Enterprise Development Teams

Large engineering organizations with 100+ daily Pull Requests can deploy this system to automate the first line of code review entirely. Senior developers are freed from routine review tasks and can focus exclusively on architecture-level and design-level decisions. The modular n8n design allows custom agent prompts to be tuned to the organization's specific coding standards and tech stack.

6.3.2 Academic and Educational Environments

Faculty at universities and coding bootcamps can deploy the system in GitHub Classroom environments to automatically review student code submissions. Students receive instant, consistent, AI-powered feedback on every PR, improving learning velocity and reducing instructor grading burden. The system can be configured to encourage learning by providing explanatory comments rather than just identifying errors.

6.3.3 Startup CI/CD Pipelines

Startups with lean engineering teams and no dedicated code reviewers can use this system as a virtual tech lead available 24 hours a day, 7 days a week. It enforces code quality standards automatically on every PR without requiring dedicated reviewer availability, enabling rapid development without quality regression.

6.3.4 DevSecOps and Security Auditing

By customizing the Agent 1 and Agent 2 prompt configurations to focus on security vulnerabilities, teams can deploy the system as a DevSecOps tool that automatically flags hardcoded credentials, SQL injection patterns, insecure API calls, and missing authentication checks on every PR before it reaches the main branch.

6.3.5 Open Source Project Maintenance

Open source project maintainers frequently face reviewer burnout from the volume of community-contributed PRs. This system can auto-review community PRs before human triaging, providing initial feedback to contributors immediately and allowing maintainers to focus their limited time on the highest-priority issues.

6.3.6 QA and Testing Teams

The system can be configured to automatically check whether new PRs include test coverage, flag missing error handling, identify uncovered edge cases, and ensure that new functions are accompanied by corresponding unit tests, significantly reducing the QA team's manual verification workload.

6.4 Inference

The N8N Multi-Agent AI Code Reviewer successfully demonstrates that multi-agent LLM orchestration, when combined with a structured workflow automation platform and human oversight, can deliver a practical, scalable, and cost-effective solution to one of the most persistent bottlenecks in modern software development: the code review process. The system achieves its objectives with 100% test case success, sub-45-second performance, and zero

direct infrastructure cost, making it viable for organizations of all sizes and technical maturity levels.

CHAPTER 7: CONCLUSIONS AND RECOMMENDATION

7.1 Outline

This chapter summarizes the conclusions drawn from the design, implementation, and evaluation of the N8N Multi-Agent AI Code Reviewer, discusses the limitations and constraints of the current system, and presents recommendations for future enhancement and research.

7.2 Conclusions

This project successfully designed and implemented a novel multi-agent AI system for the automated review of GitHub Pull Requests, built entirely within the n8n workflow automation platform using free-tier Large Language Models. The following conclusions are drawn from the work:

- Multi-agent AI architectures consistently outperform single-model approaches for code review tasks, as demonstrated by Agent 2's ability to identify issues missed by Agent 1 in the majority of complex test cases.
- The pre-filter safety mechanism is essential for production deployments, reducing unnecessary LLM API consumption and ensuring that lightweight PRs receive proportionally lightweight processing.
- Human-in-the-loop approval gates are a practical and effective mechanism for bridging the gap between AI automation and human accountability in software quality workflows.
- The n8n platform provides a capable and accessible orchestration layer for multi-agent AI pipelines, enabling visual design, rapid iteration, and deployment without requiring custom backend infrastructure.
- The system proves that high-quality automated code review is achievable at zero direct compute cost using publicly available free-tier LLM APIs, democratizing access to AI-assisted development tooling.

7.3 Limitations and Constraints

- Although the system performs effectively, there are some limitations that need to be considered.
- For very large Pull Requests, the response time may increase since multiple AI agents need to process the entire code. In some cases, the models may not analyze all files completely due to input size limitations.
- The system also depends on external APIs, so any changes in availability or pricing may affect its performance. Additionally, AI-generated reviews may sometimes include false positives, which is why human approval remains important.
- Another limitation is that the system does not store past reviews, meaning it cannot learn from previous feedback or improve over time.

7.4 Future Enhancements

- There are several ways this system can be improved in the future.
- Integration with platforms like Slack or Microsoft Teams can make the approval process faster and more convenient. The system can also be enhanced by training models on specific codebases to improve accuracy for particular projects.
- Adding integration with tools like Jira can help automatically create tasks for identified issues. A severity-based system can also be introduced to prioritize feedback based on importance.
- Performance can be improved by running multiple AI agents in parallel instead of sequentially. Additionally, storing past reviews and using them for learning can help improve the system over time.

7.5 Summary

The N8N Multi-Agent AI Code Reviewer represents a meaningful contribution to the field of AI-assisted software engineering. It demonstrates a novel, practical, and cost-effective approach to automated code review that addresses the limitations of all existing tools by combining multi-agent LLM consensus, intelligent pre-filtering, heterogeneous model orchestration, and human-supervised automation. The system is production-deployable,

extensible, and applicable across a wide range of software development environments. The identified limitations provide clear directions for future research and enhancement.

REFERENCES

[1] Z. Geng, S. Liu, and F. Ma, "Automated Code Review using Large Language Models," in Proc. IEEE International Conference on Software Engineering (ICSE), 2023, pp. 112–123.

[2] Y. Liu, X. Wang, and J. Chen, "LLM-Based Pull Request Description Generation," ACM Transactions on Software Engineering and Methodology, vol. 32, no. 4, pp. 1–28, 2023.

[3] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, G. Xu, Z. Liu, and M. Sun, "Communicative Agents for Software Development," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2023.

[4] M. Amershi, A. Cakmak, W. B. Knox, and T. Kulesza, "Power to the People: The Role of Humans in Interactive Machine Learning," AI Magazine, vol. 35, no. 4, pp. 105–120, 2014.

[5] n8n GmbH, "n8n Workflow Automation Platform Documentation," [Online]. Available: https://docs.n8n.io.

[6] OpenRouter, "OpenRouter API Documentation: Multi-Model LLM API Gateway," [Online]. Available: https://openrouter.ai/docs.

[7] NVIDIA, "Nemotron Language Models," [Online]. Available: https://www.nvidia.com/en-us/ai/.

[8] Alibaba Cloud, "Qwen-Coder: Code-Specialized Large Language Model," [Online]. Available: https://qwenlm.github.io.

[9] GitHub, "GitHub REST API Documentation: Pull Requests," [Online]. Available: https://docs.github.com/en/rest/pulls.

[10] T. Zimmermann, R. Premraj, N. Bettenburg, S. Just, A. Schroter, and C. Weiss, "What Makes a Good Bug Report?" IEEE Transactions on Software Engineering, vol. 36, no. 5, pp. 618–643, 2010.

APPENDIX A: AGENT PROMPT REFERENCE

A.1 Complete Agent Prompt Specifications

Agent 1 — First Reviewer (Nemotron-3-Nano via OpenRouter):

System: "You are a senior web developer with 10+ years of experience. Review the provided code diff file by file. Generate detailed, constructive inline comments for each file. Identify: syntax errors, runtime bugs, missing null/undefined checks, anti-patterns, hardcoded values, missing error handling, inefficient algorithms, and security risks. For each issue, suggest a specific fix. Format your response with file names as headers."

Agent 2 — Complementary Reviewer (Nemotron-3-Nano via OpenRouter):

System: "You are a senior code reviewer providing a second opinion on a diff that has already been reviewed by Agent 1. Your task is ONLY to find issues, suggestions, and improvements that Agent 1 has NOT already covered. Do not repeat any point already made. Format: 'AI Agent 1: {Agent 1 full output} AI Agent 2: {your unique findings only}'."

Agent 3 — Combiner (Qwen3-Coder via OpenRouter):

System: "You are a technical editor. Combine the two code review outputs provided (from Agent 1 and Agent 2) into a single, coherent, deduplicated technical review. Remove all redundant points. Resolve any contradictions in favor of the more specific finding. Add your own observations if warranted. Produce one clean, comprehensive technical code review."

Agent 4 — Safety Guard / Humanizer (Nemotron-3-Nano via OpenRouter):

Safety Guard Mode System: "You are a senior developer screening a code diff for significance. If the diff contains ONLY documentation updates, README changes, list modifications, or formatting changes with no logic impact, output exactly and only: APPROVE. Otherwise, output: REVIEW NEEDED."

Humanizer Mode System: "Rewrite the following technical code review as a professional, friendly, 3-to-4 line GitHub Pull Request comment. Remove all technical jargon, AI-specific phrasing, and lengthy explanations. The output must read as if a senior developer wrote it directly and personally."

APPENDIX B: N8N WORKFLOW CONFIGURATION SUMMARY

B.1 Workflow Node Count and Configuration

The complete n8n workflow for the N8N Multi-Agent AI Code Reviewer consists of 22 nodes organized across 7 logical phases as described in Chapter 5. Below is a summary of the key node types and their configurations:

#	Node Name	Type	Key Configuration
1	GitHub Trigger	GitHub Trigger Node	Event: pull_request; Actions: opened, synchronize
2	Fetch File Diffs	HTTP Request Node	GET /repos/{owner}/{repo}/pulls/{number}/files; Auth: Bearer
3	Format Diff Prompt	Code Node (JS)	Iterates files, sanitizes patches, assembles LLM prompt
4	Safety Guard (Agent 4)	LLM Agent Node	Model: Nemotron via OpenRouter; Role: Pre-filter check
5	If1 — Trivial/Complex	If Node	Condition: output contains 'APPROVE'
6	Agent 1 (First Review)	LLM Agent Node	Model: Nemotron; Role: First-pass
7	Agent 2 (Second Review)	LLM Agent Node	Model: Nemotron; Role: Complementary reviewer
8	Agent 3 (Combiner)	LLM Agent Node	Model: Qwen-Coder; Role: Merge and deduplicate

9	Agent 4 (Humanizer)	LLM Agent Node	Model: Nemotron; Role: Natural-language rewrite
10	Gmail Send & Wait	Gmail Node	Send review draft + Yes/No approval form; wait for response
11	If2 — Approved/ Rejected	If Node	Condition: approval response == 'Yes'
12	Post GitHub Review	GitHub Node	Create PR review comment with approved text
13	Label Issue Skipped	GitHub Node	Edit issue: add label 'review-skipped'

B.2 GitHub Repository

The test repository used for validating this project is publicly available at:

https://github.com/Parth-Gholap/test-ai-reviewer

This repository contains sample PRs used for testing all 10 test cases described in Chapter 5, along with example AI-generated review comments for reference.